

Basic Building Blocks

Course: Rust Basics

Author: Aleksandr Kugushev

Structs

```
struct MyStruct {  
    value1: i32,  
    value2: bool,  
}
```

```
let my_struct = MyStruct {  
    value1: 42,  
    value2: true,  
};
```

Rust Structs vs “Java” Classes

Rust Structs

Java Classes

```
struct MyStruct {  
    num: i32  
}  
  
struct MyOtherStruct {  
    str: MyStruct  
}
```

Structs are always mutable

```
struct MyStruct {  
    readonly value1: i32,  
    value2: bool,  
}
```

No Constructors

```
fn new(value1: i32, value2: bool) -> MyStruct {  
    MyStruct {  
        value1: value1,  
        value2: value2  
    }  
}
```

```
fn new(value1: i32, value2: bool) -> MyStruct {  
    MyStruct {  
        value1,  
        value2  
    }  
}
```

Struct Update Syntax

```
struct MyBigStruct  
{  
    value1: i32,  
    value2: i32,  
    value3: i32,  
    value4: i32,  
    value5: i32,  
    value6: i32,  
    value7: String,  
}
```

```
let var1 = MyBigStruct{  
    value1: 1, value2: 2, value3: 3,  
    value4: 4, value5: 5, value6: 6,  
    value7: String::from("Hello World"),  
};  
let var2 = MyBigStruct{  
    value1: 42,  
    ..var1  
};
```

Tuples

```
let tuple: (i32, bool, String) = (42, true, String::from("Hi"));
```

```
let number = tuple.0;
```

```
let (num, bool, str) = tuple;
```

```
let str = StrWithTuple { field: tuple };
```

```
struct StrWithTuple {  
    field: (i32, bool, String),  
}
```

Tuple with single value

```
let tuple: (i32) = (42);
```

```
let int = tuple.0; error[E0610]: `i32` is a primitive type and therefore doesn't  
have fields
```

```
let not_tuple: i32 = (42);
```


Why do we need tuples?

- Multiple return
- ...
- ...

Return tuple from method

```
fn go_like_function(age: i32) -> (bool, String) {  
    if age > 0 && age < 120 {  
        (true, String::from("You are alive!"))  
    } else {  
        (false, String::from("You are not alive!"))  
    }  
}  
  
let (error, message) = go_like_function(42);  
if !error {  
    eprint!("Error: {}", message);  
}
```

Tuple Structs

```
struct TupleStruct(i32, i32);
```

```
let point = TupleStruct(42, 666);
```

Array

```
let array: [i32; 5] = [1, 2, 3, 4, 5];
```

```
let var1 = array[4];
```

```
let var2 = array[42]; <- error: this operation will panic at runtime
```

```
struct MyStructWithArray {  
    array: [i32; 5],  
}
```

Structs vs Tuple Structs vs Tuples

- Struct
 - Own Data
 - Different Data
 - Has Name
 - Named Fields
- Tuple Struct
 - Own Data
 - Different Data
 - Has Name
- Tuple
 - Own Data
 - Different Data
- Array
 - Own Data
- Unit

Unit

```
let unit = ();
```

```
let unit: () = while false {};
```

!

```
let bang: ! = panic!();
```

```
let bang: ! = loop {};
```

```
let bang: ! = unreachable!();
```

```
let bang: ! = unimplemented!();
```

Structs vs Tuple Structs vs Tuples

- Struct
 - Own Data
 - Different Data
 - Has Name
 - Named Fields
- Tuple Struct
 - Own Data
 - Different Data
 - Has Name
- Tuple
 - Own Data
 - Different Data
- Array
 - Own Data
- Unit
- Bang
- Zero Sized Struct
 - Has Name

Zero Sized Struct

```
struct MyZeroSizedStruct;
```

```
std::mem::size_of::<MyZeroSizedStruct>()
```

```
>> 0
```

```
let zst_arr = [MyZeroSizedStruct; 10];
```

```
std::mem::size_of_val(&zst_arr)
```

```
>> 0
```

Practice

Game character

- Name
- Position
- Arrows count by type: fire, ice, poison
- HP Potions (different fullness) (max 5)

Methods

```
struct MyStruct {  
    num: i32  
}
```

```
my_struct = my_struct.update_and_print(10);
```

```
impl MyStruct {  
    fn update_and_print(mut self, increment: i32) -> Self {  
        self.num += increment;  
        println!("{}", self.num);  
        self  
    }  
}
```

Keyword SELF

```
struct MyStruct {  
    num: i32  
}
```

```
my_struct = my_struct.update_and_print(10);
```

```
impl MyStruct {  
    fn update_and_print(mut self, increment: i32) -> Self {  
        self.num += increment;  
        println!("{}", self.num);  
        self  
    }  
}
```

Desugaring Methods

```
my_struct.update_and_print(10)  
;  
MyStruct::update_and_print(my_struct, 10);
```

Impl block

```
impl MyStruct {  
    fn any_static_method() {}  
}
```

```
impl MyStruct {  
    fn any_static_method() {}  
}
```

Associated Functions

```
impl MyStruct {  
    fn static_method() {  
        println!("My Struct is good");  
    }  
}
```

```
    fn update_and_print(mut self,  
increment: i32) -> Self {  
        self.num += increment;  
        println!("{}", self.num);  
        self  
    }  
}
```

```
MyStruct::static_method();
```

```
my_struct.update_and_print(10  
);
```

Ctor

```
struct MyStruct {  
    num: i32  
}
```

```
impl MyStruct {  
    fn new(num: i32) ->  
    Self {  
        Self { num }  
    }  
}
```

```
let mut my_struct = MyStruct::new(42);  
my_struct.update_and_print(10);  
MyStruct::static_method();
```


Impl block

```
impl String {  
    fn any_static_method() {}  
}
```

error[E0116]: cannot define inherent `impl` for a type outside of the crate where the type is defined

Attributes

```
#[test]
fn test_foo() {
    /* ... */
}

fn some_unused_variables() {
    #![allow(unused_variables)]
}
```

Traits

```
trait MyTrait{  
    fn print_and_clean(self) -> Self;  
}
```

```
impl MyTrait for MyStruct {  
    fn print_and_clean(mut self) -> Self {  
        println!("{}", self.num);  
        self.num = 0;  
        self  
    }  
}
```

Traits for everything

```
impl MyTrait for (bool, i32) {  
    fn print_and_clean(mut self) ->  
Self {  
    println!("{}", self.1);  
    self.0 = false;  
    self.1 = 0;  
    self  
    }  
}  
(true, 42).print_and_clean();
```

```
impl MyTrait for [MyStruct; 2] {  
    fn print_and_clean(mut self) ->  
Self {  
    self[0].print_and_clean();  
    self[1].print_and_clean();  
    self  
    }  
}  
[MyStruct::new(42),  
MyStruct::new(42)].print_and_clean()  
;
```

Supertrait

```
trait Trait: Supertrait {}  
trait Supertrait {}
```

```
fn upcast(x: &dyn Trait) -> &dyn Supertrait {  
    x  
}
```

Traits

- Default Implementation
- Static Polymorphism
- Dynamic Polymorphism

Implementation

```
trait AnswerOnTheQuestion {  
    fn answer_on_the_question() -> i32;  
}
```

```
impl AnswerOnTheQuestion for MyStruct {  
    fn answer_on_the_question() -> i32 {  
        42  
    }  
}
```

Default Implementation

```
trait AnswerOnTheQuestion {  
    fn answer_on_the_question() -> i32 {  
        42  
    }  
}  
  
impl AnswerOnTheQuestion for MyStruct {  
}
```


Useful Traits with Default Implementation

```
#[derive(...)]  
struct MyLuckyStruct {  
    value1: i32,  
    value2: bool,  
}
```

- Default
- Copy
- Clone

Default

```
#[derive(Default)]  
struct MyLuckyStruct {  
    value1: i32,  
    value2: bool,  
}
```

```
let default_str = MyLuckyStruct::default();
```

Default

```
#[derive(Default)]  
struct MyLuckyStruct {  
    value1: i32,  
    value2: bool,  
    value3: MyStruct  
}
```

<- the trait `Default` is not implemented for `MyStruct`

error[E0277]: the trait bound ``MyStruct: Default`` is not satisfied

Clone

```
#[derive(Clone, Default)]  
struct MyLuckyStruct {  
    value1: i32,  
    value2: bool  
}
```

```
let default_str = MyLuckyStruct::default();  
let another_str = default_str.clone();  
println!("{}", default_str.value1);  
println!("{}", another_str.value1);
```

Copy

```
#[derive(Copy, Clone, Default)]
struct MyLuckyStruct {
    value1: i32,
    value2: bool
}

let default_str = MyLuckyStruct::default();
let another_str = default_str;
println!("{}", default_str.value1);
println!("{}", another_str.value1);
```

Default Implementation is not mandatory

```
impl Default for MyStruct {  
    fn default() -> Self {  
        Self { value1: 0, value2: false }  
    }  
}
```

Third-party traits and impls

My Crate

```
trait MyTrait {}  
struct MyStruct ;
```

```
impl MyTrait for OtherStruct {}  
impl OtherTrait for MyStruct {}
```

```
impl OtherTrait for OtherStruct {}  
impl OtherTrait for i32 {}
```

impl doesn't use only types from inside the current crate

Third party crate

```
trait OtherTrait {}  
struct OtherStruct;
```

Mix

```
trait Power {  
    fn amount(self) -> i32;  
    fn is_powerful(self) -> bool {  
        self.amount() > 9000  
    }  
}
```

```
impl Power for i32 {  
    fn amount(self) -> i32 {  
        self  
    }  
}
```


Mix

```
trait Power {  
    fn amount(self) -> i32;  
    fn is_powerful(self) -> bool {  
        self.amount() > 9000  
    }  
}
```

```
impl Power for bool {  
    fn amount(self) -> i32 {  
        if self { i32::MAX } else  
        { i32::MIN }  
    }  
  
    fn is_powerful(self) -> bool {  
        self  
    }  
}
```

Trait as extension method

```
struct Vec2(i32, i32);
```

```
trait Convert {  
    fn to_vec2(self) -> Vec2;  
}
```

```
impl Convert for i32 {  
    fn to_vec2(self) -> Vec2 {  
        Vec2(self, self)  
    }  
}
```

```
let vec = 42.to_vec2();
```

Trait as extension method

```
struct Vec2(i32, i32);
```

```
trait Convert {  
    fn to_vec2(self) -> Vec2;  
}
```

```
impl Convert for (i32, i32) {  
    fn to_vec2(self) -> Vec2 {  
        Vec2(self.0, self.1)  
    }  
}
```

```
let vec = (42, 42).to_vec2();
```

Trait as extension method

```
struct Vec2(i32, i32);
```

```
trait Convert {  
    fn to_vec2(self) -> Vec2;  
}
```

```
impl Convert for [i32; 2] {  
    fn to_vec2(self) -> Vec2 {  
        Vec2(self[0], self[1])  
    }  
}
```

```
let vec = [42, 42].to_vec2();
```

Why do we need tuples?

- Multiple return
- Traits
- ...

Tuples Traits

12 items

- PartialEq
- Eq
- PartialOrd
- Ord
- Debug
- Default
- Hash
- From<[T; N]>

Any length

- Clone
- Copy
- Send
- Sync
- Unpin
- UnwindSafe
- RefUnwindSafe

Homework

- Rustlings (<https://github.com/rust-lang/rustlings/tree/main/exercises/>):
 - 07_structs
 - 15_traits
- Courseware
 - Go through files recursively
 - Add "Find" option: cargo run -- c:/Projects --find main.rs
 - Tips:
 - Don't use features, that you doesn't know
 - Use different structs and traits to follow DRY principle
 - Don't forget about Copy, Clone traits: it's ok for now